

Rapport détaillé — Phase de déploiement du projet Pi-CloudDOOM

1. Introduction générale

La phase de déploiement du projet **Pi-CloudDOOM** a représenté une étape longue, complexe et très importante dans l'évolution du projet. Après le développement des différents microservices, de l'interface frontend, de l'authentification, des fonctionnalités IA et des intégrations externes, l'objectif principal était de transformer l'application en une solution réellement exécutable dans un environnement cloud, proche d'un environnement de production.

Cette phase ne s'est pas limitée à lancer quelques conteneurs. Elle a inclus la préparation de l'infrastructure, la création et la stabilisation d'un cluster Kubernetes, la conteneurisation des services, la configuration des variables d'environnement, la gestion du réseau, l'exposition publique de l'application, la correction des problèmes d'authentification OAuth, l'intégration des services externes comme Azure AI/Ollama, la mise en place d'un système de monitoring, ainsi que la réflexion autour de la détection d'attaques.

L'objectif de ce rapport est de résumer de manière complète et compréhensible tout ce qui a été réalisé pendant cette phase de déploiement, en expliquant les technologies utilisées, les stratégies adoptées, les problèmes rencontrés, les solutions appliquées et les choix techniques effectués.

2. Présentation du projet Pi-CloudDOOM

Pi-CloudDOOM est une application web basée sur une architecture **microservices**. Le projet combine plusieurs domaines techniques : développement backend, frontend, sécurité, cloud computing, conteneurisation, orchestration, intelligence artificielle, monitoring et DevOps.

L'application repose sur plusieurs services indépendants, chacun responsable d'un domaine fonctionnel spécifique. Cette séparation permet de mieux organiser le code, de faciliter le déploiement, de rendre chaque service évolutif indépendamment et de rapprocher le projet d'une architecture moderne utilisée en entreprise.

2.1 Objectifs fonctionnels du projet

Le projet vise à fournir une plateforme complète contenant plusieurs modules, notamment :

- gestion des utilisateurs ;
- authentification et autorisation ;
- connexion via fournisseurs OAuth comme Google, LinkedIn et GitHub ;
- gestion des profils utilisateurs ;
- upload et consultation de CV ;
- extraction et enrichissement automatique du profil à partir d'un CV ;

- services communautaires ;
- services d'entretien/interview ;
- services liés à la formation ;
- communication entre services ;
- interface web Angular ;
- fonctionnalités assistées par intelligence artificielle.

2.2 Objectifs techniques de la phase de déploiement

La phase de déploiement avait plusieurs objectifs principaux :

1. **Préparer une infrastructure cloud privée** basée sur OpenStack.
2. **Créer un cluster Kubernetes** capable d'héberger les microservices.
3. **Déployer les services backend et frontend** sous forme de conteneurs.
4. **Configurer la communication entre les services.**
5. **Exposer l'application publiquement** à travers un domaine et Cloudflare.
6. **Corriger les erreurs liées à OAuth et aux URLs de redirection.**
7. **Intégrer l'IA de manière réaliste** malgré les limitations de ressources.
8. **Mettre en place une base de monitoring.**
9. **Préparer une stratégie de détection d'attaques.**
10. **Documenter toute la démarche** afin que le déploiement soit compréhensible, reproductible et défendable devant un jury ou un professeur.

3. Architecture générale du projet

Le projet suit une architecture distribuée basée sur plusieurs composants.

3.1 Architecture applicative

L'application est composée d'un frontend Angular et de plusieurs microservices backend développés avec Spring Boot. Chaque microservice est conteneurisé et déployé séparément.

Les principaux composants sont :

- **Frontend Angular** : interface utilisateur principale.
- **User Service** : gestion des utilisateurs, profils, CV, authentification côté métier et intégration IA liée au CV.
- **Community Service** : gestion des fonctionnalités communautaires.
- **Interview Service** : gestion des fonctionnalités liées aux entretiens.
- **Training Service** : gestion des formations.
- **Keycloak** : serveur d'identité et d'authentification.
- **PostgreSQL** : base de données relationnelle.
- **Redis** : cache et stockage temporaire.
- **Kafka** : communication asynchrone entre services.
- **Ollama / Azure AI** : traitement IA pour l'analyse du CV.
- **Kubernetes** : orchestration des conteneurs.

- **Cloudflare** : DNS, exposition publique et sécurisation de l'accès.
- **Monitoring stack** : observation de l'état de l'application et du cluster.

3.2 Logique microservices

L'architecture microservices permet de découper l'application en plusieurs services autonomes. Chaque service possède son propre rôle, son propre cycle de vie et ses propres variables de configuration.

Ce choix apporte plusieurs avantages :

- meilleure séparation des responsabilités ;
- maintenance plus simple ;
- possibilité de déployer un service sans redéployer toute l'application ;
- meilleure scalabilité ;
- meilleure organisation en travail d'équipe ;
- alignement avec les pratiques DevOps et cloud-native.

Cependant, cette architecture augmente aussi la complexité du déploiement. Il faut gérer :

- les ports ;
- les variables d'environnement ;
- les noms DNS internes Kubernetes ;
- les secrets ;
- les dépendances entre services ;
- les logs ;
- les erreurs de communication ;
- la configuration OAuth ;
- les images Docker ;
- les probes Kubernetes ;
- la persistance des données.

4. Technologies utilisées

4.1 Backend — Spring Boot

Les microservices backend sont développés avec **Spring Boot**. Spring Boot a été choisi car il permet de créer rapidement des APIs REST robustes, bien structurées et faciles à intégrer avec des bases de données, des systèmes de sécurité et des outils cloud.

Les éléments importants utilisés dans Spring Boot incluent :

- Spring Web pour les APIs REST ;
- Spring Security pour la sécurisation des endpoints ;
- OAuth2 Resource Server pour valider les JWT émis par Keycloak ;
- Spring Data JPA pour l'accès aux données ;
- configuration par `application.properties` ou variables d'environnement ;

- endpoints de santé via Actuator dans certains cas ;
- intégration avec services externes comme Ollama ou Azure AI.

4.2 Frontend — Angular

Le frontend est développé avec **Angular**. Il représente l'interface principale de l'utilisateur et communique avec les microservices backend à travers des APIs HTTP.

Pendant la phase de déploiement, il a fallu s'assurer que le frontend utilise les bonnes URLs d'API, notamment lorsqu'il n'est plus exécuté en local mais dans un environnement distant/public.

Les points importants côté Angular :

- configuration des endpoints backend ;
- intégration des boutons OAuth ;
- gestion des redirections après login ;
- affichage du profil utilisateur ;
- affichage du CV uploadé ;
- consommation des données enrichies par l'IA ;
- gestion des erreurs côté interface.

4.3 Docker

Docker a été utilisé pour conteneuriser les services. Chaque service possède une image Docker qui contient son code, ses dépendances et sa configuration d'exécution.

Docker a permis :

- d'uniformiser les environnements ;
- d'éviter les problèmes liés aux dépendances locales ;
- de préparer les services pour Kubernetes ;
- de pousser les images vers Docker Hub ;
- de faciliter le rollback en changeant simplement le tag d'image.

Des images Docker ont été construites pour les services, par exemple :

- `user-service` ;
- `community-service` ;
- `interview-service` ;
- autres services Spring Boot ;
- frontend Angular si déployé sous forme de conteneur.

4.4 Docker Hub

Docker Hub a été utilisé comme registre d'images. Les images construites localement ou depuis l'environnement de développement ont été taguées puis poussées vers Docker Hub afin que Kubernetes puisse les télécharger.

Un exemple d'image utilisée pendant la phase de correction était :

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

Ce type d'image a été utilisé dans les manifests Kubernetes pour lancer les pods correspondants.

4.5 Kubernetes

Kubernetes est l'outil principal d'orchestration utilisé pour déployer l'application. Il permet de gérer les conteneurs sous forme de ressources déclaratives.

Les ressources Kubernetes utilisées ou manipulées incluent :

- Namespace ;
- Deployments ;
- Pods ;
- Services ;
- ConfigMaps ;
- Secrets ;
- Persistent Volumes et Persistent Volume Claims ;
- StorageClass ;
- Ingress ou exposition via services selon les cas ;
- Rollout ;
- Rollback ;
- logs et descriptions de pods.

Le namespace principal utilisé était :

```
piclouddoom
```

Ce namespace a permis d'isoler les ressources du projet des autres ressources du cluster.

4.6 OpenStack

OpenStack a été utilisé comme infrastructure cloud privée. Les machines virtuelles nécessaires au cluster Kubernetes ont été créées sur OpenStack.

OpenStack a fourni :

- les instances virtuelles ;
- le réseau ;
- les volumes ;
- les adresses IP ;
- les ressources CPU/RAM ;
- la gestion des snapshots ;
- la possibilité de recréer le cluster depuis zéro.

Cette partie a demandé beaucoup d'attention, car les ressources étaient limitées et certains problèmes système sont apparus, notamment autour des volumes, du boot des instances, des clés SSH et de la disponibilité des nœuds.

4.7 Cloudflare

Cloudflare a été utilisé pour gérer l'accès public à l'application, notamment via le DNS et la sécurisation de l'accès HTTP/HTTPS.

Cloudflare a servi à :

- pointer un domaine vers l'infrastructure ;
- gérer les problèmes de résolution DNS ;
- exposer l'application à l'extérieur ;
- sécuriser partiellement l'accès ;
- diagnostiquer des erreurs comme l'erreur Cloudflare 1033.

4.8 Keycloak

Keycloak a été utilisé comme serveur d'identité. Il gère l'authentification, les rôles, les tokens JWT et les connexions via fournisseurs externes.

La configuration Keycloak comprenait :

- un realm dédié au projet ;
- des clients OAuth ;
- des rôles comme `ROLE_USER`, `ROLE_ADMIN`, `ROLE_MANAGER` ;
- des redirections autorisées ;
- des fournisseurs d'identité externes : Google, LinkedIn, GitHub ;
- la validation des tokens par les microservices Spring Boot.

La configuration OAuth a été l'une des parties les plus sensibles du déploiement, car elle dépend fortement des URLs publiques, des redirect URIs et de la cohérence entre Keycloak, le frontend et les fournisseurs externes.

4.9 PostgreSQL

PostgreSQL a été utilisé comme base de données relationnelle. Il stocke les données métier des services, notamment les utilisateurs, les profils et d'autres entités applicatives.

Dans le déploiement Kubernetes, PostgreSQL nécessite une attention particulière concernant :

- la persistance des données ;
- les credentials ;
- les variables d'environnement ;
- la disponibilité du service ;
- les volumes persistants.

4.10 Redis

Redis a été utilisé comme système de cache ou de stockage temporaire. Il permet d'améliorer les performances et de gérer certaines données rapides à consulter.

Dans Kubernetes, Redis est généralement déployé avec un service interne accessible par les autres pods.

4.11 Kafka

Kafka a été utilisé pour la communication asynchrone entre microservices. Ce type de communication permet de découpler les services : un service peut publier un événement sans attendre directement la réponse d'un autre service.

Kafka est utile pour :

- les événements utilisateur ;
- les notifications ;
- la communication interservices ;
- la scalabilité ;
- la résilience.

4.12 Ollama et Azure AI

Le projet a intégré une fonctionnalité IA pour l'analyse de CV. Plusieurs stratégies ont été étudiées :

- utilisation initiale de Gemini ;
- migration vers Ollama pour exécuter un modèle localement ;
- réflexion sur l'utilisation d'Azure pour exécuter Ollama ou un service IA séparé ;
- optimisation des coûts avec Azure Container Apps ;
- possibilité d'utiliser une stratégie hybride pour éviter de surcharger le cluster OpenStack.

Le modèle IA était utilisé principalement pour extraire ou enrichir les informations du profil utilisateur à partir d'un CV PDF.

5. Infrastructure OpenStack et préparation du cluster

5.1 Création des machines virtuelles

La première grande étape du déploiement a consisté à créer les machines virtuelles sur OpenStack. Ces machines devaient ensuite former le cluster Kubernetes.

L'objectif initial était d'avoir un cluster suffisamment robuste pour héberger plusieurs microservices. Une architecture avec plusieurs nœuds a été envisagée, par exemple :

- un ou plusieurs control planes ;
- plusieurs workers ;

- répartition des workers sur différents compute nodes ;
- utilisation de volumes Cinder pour certains besoins persistants.

Les instances ont été créées avec des ressources adaptées aux limites de l'environnement disponible.

5.2 Gestion des ressources limitées

L'un des défis majeurs était la limitation des ressources OpenStack. Le cluster devait faire tourner plusieurs composants gourmands :

- microservices Spring Boot ;
- frontend ;
- Keycloak ;
- PostgreSQL ;
- Redis ;
- Kafka ;
- monitoring ;
- éventuellement Ollama.

À cause de ces contraintes, une stratégie hybride a été envisagée : conserver certains services dans le cluster OpenStack et externaliser d'autres composants vers Azure, Vercel ou un autre environnement plus adapté.

Cette réflexion était importante, car l'objectif n'était pas seulement de faire fonctionner le projet une fois, mais de proposer une architecture cohérente, justifiable et réaliste.

5.3 Problèmes de volumes et snapshots

Pendant la préparation de l'infrastructure, plusieurs opérations ont été réalisées autour des volumes OpenStack :

- vérification de volumes attachés ;
- suppression de volumes ;
- gestion de snapshots ;
- diagnostic de volumes bloqués ;
- utilisation de commandes OpenStack pour afficher les volumes, les attachments et les snapshots.

Un cas important concernait un volume qui ne pouvait pas être supprimé directement car il possédait un snapshot. La démarche correcte était alors :

1. vérifier que le volume n'était attaché à aucune instance ;
2. lister les snapshots associés ;
3. supprimer le snapshot ;
4. supprimer ensuite le volume.

Cette étape a permis de mieux comprendre la relation entre volumes, snapshots et instances dans OpenStack.

5.4 Problèmes SSH

Pendant la recréation ou le redéploiement de machines, une erreur fréquente est apparue :

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
```

Cette erreur se produit lorsqu'une adresse IP est réutilisée par une nouvelle machine mais que l'ancien fingerprint SSH est encore enregistré dans le fichier `known_hosts`.

La solution appliquée était de supprimer l'ancienne entrée SSH avec :

```
ssh-keygen -f ~/.ssh/known_hosts -R <IP>
```

Puis de se reconnecter normalement à la nouvelle instance.

5.5 Problèmes de boot et filesystem

Certaines instances ont rencontré des problèmes au démarrage, notamment des blocages liés au filesystem ou à l'environnement initramfs. Ces situations ont nécessité une approche prudente :

- observer la console de la VM ;
- vérifier si le système démarre correctement ;
- identifier si le problème vient du disque, du montage ou d'un volume ;
- éviter de détruire directement l'instance sans comprendre la cause ;
- utiliser des snapshots ou une recréation si nécessaire.

Ces incidents ont montré l'importance d'une bonne gestion des volumes et de la configuration système avant de lancer Kubernetes.

6. Création et stabilisation du cluster Kubernetes

6.1 Objectif du cluster

Le cluster Kubernetes devait servir d'environnement principal pour héberger les microservices du projet. Il devait permettre :

- l'exécution stable des pods ;
- la communication interne entre services ;
- l'exposition externe de l'application ;
- la persistance des données ;
- le monitoring ;
- la possibilité de redéployer rapidement en cas d'erreur.

6.2 Nœuds du cluster

Le cluster était composé de plusieurs machines virtuelles, avec notamment :

- un nœud control plane, par exemple `k8s-cp1` ;
- plusieurs workers, par exemple `k8s-w1`, `k8s-w2`, `k8s-w3`, `k8s-w4` .

Le control plane pilote le cluster et les workers exécutent les pods applicatifs.

6.3 Réseau Kubernetes

Pour que les pods puissent communiquer entre eux, un CNI devait être installé. Calico a été envisagé ou utilisé comme plugin réseau.

Le rôle du CNI est fondamental :

- attribuer des IPs aux pods ;
- permettre la communication pod-to-pod ;
- gérer les routes réseau internes ;
- appliquer éventuellement des politiques réseau.

6.4 Namespace du projet

Un namespace dédié a été utilisé :

```
piclouddoom
```

L'utilisation d'un namespace permet :

- de regrouper les ressources du projet ;
- de simplifier les commandes `kubectl` ;
- d'éviter les conflits avec d'autres applications ;
- de faciliter le nettoyage ou le redéploiement.

Exemple de commande utilisée :

```
kubectl get pods -n piclouddoom
```

6.5 StorageClass et local-path-provisioner

Un problème important dans Kubernetes concernait la persistance. Au départ, aucune StorageClass n'était disponible :

```
kubectl get storageclass
```

renvoyait aucun résultat.

Pour résoudre cela, `local-path-provisioner` a été installé :

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-  
provisioner/master/deploy/local-path-storage.yaml  
kubectl get storageclass
```

Ce provisioner permet de créer des volumes persistants locaux sur les nœuds Kubernetes. Cette solution est simple et efficace pour un projet académique ou un environnement de démonstration.

Elle permet notamment à des services comme PostgreSQL ou Keycloak de conserver leurs données entre les redémarrages de pods.

7. Stratégie de déploiement Kubernetes

7.1 Déploiement par manifests YAML

Les services ont été déployés à l'aide de fichiers YAML Kubernetes. Chaque service nécessite généralement :

- un Deployment ;
- un Service ;
- parfois un ConfigMap ;
- parfois un Secret ;
- parfois un PersistentVolumeClaim.

Le Deployment définit comment lancer les pods, avec quelle image Docker, quelles variables d'environnement, quels ports et quelles probes.

Le Service permet d'exposer le pod à l'intérieur du cluster ou vers l'extérieur.

7.2 Déploiements backend

Chaque microservice Spring Boot a été déployé dans Kubernetes sous forme de Deployment.

Un déploiement typique contient :

- le nom du service ;
- le nombre de replicas ;
- l'image Docker ;
- le port exposé ;

- les variables d'environnement ;
- les dépendances vers PostgreSQL, Redis, Kafka ou Keycloak ;
- des probes de santé si disponibles.

Exemple de vérification :

```
kubectl get deploy user-service -n piclouddoom -o wide
kubectl get pods -n piclouddoom | grep user-service
```

7.3 Déploiement du frontend

Le frontend Angular devait être configuré pour communiquer avec les URLs correctes des services backend. En local, l'application utilise souvent `localhost`, mais en production ou en Kubernetes, `localhost` ne fonctionne plus de la même manière.

Il fallait donc adapter :

- les URLs des APIs ;
- les URLs OAuth ;
- les redirections après login ;
- le domaine public ;
- les chemins servis par le frontend.

7.4 Déploiement de Keycloak

Keycloak est un composant critique. Il doit être disponible et correctement configuré pour que l'authentification fonctionne.

Les points essentiels de Keycloak dans le déploiement étaient :

- realm du projet ;
- client Angular ;
- clients backend si nécessaire ;
- redirect URIs ;
- web origins ;
- identity providers ;
- rôles ;
- URLs publiques ;
- issuer URI utilisé par Spring Security.

Une mauvaise configuration Keycloak peut provoquer :

- des erreurs 401 ;
- des erreurs OAuth ;
- des redirections invalides ;
- des erreurs de callback ;
- des tokens non acceptés par les services backend.

7.5 Déploiement de PostgreSQL

PostgreSQL devait être déployé avec persistance. Sans volume persistant, les données seraient perdues à chaque redémarrage du pod.

Les éléments importants :

- PersistentVolumeClaim ;
- StorageClass ;
- variables d'environnement pour user, password et database ;
- service interne pour permettre aux microservices d'accéder à PostgreSQL ;
- vérification des logs en cas d'erreur de connexion.

7.6 Déploiement de Redis

Redis a été déployé comme service interne. Les microservices qui en dépendent doivent utiliser le nom DNS Kubernetes du service Redis au lieu de `localhost`.

7.7 Déploiement de Kafka

Kafka est plus complexe à déployer que Redis ou PostgreSQL, car il nécessite une bonne configuration réseau, notamment autour des listeners et advertised listeners.

Dans un cluster Kubernetes, Kafka doit être configuré pour que les services puissent le contacter via un nom DNS interne stable.

7.8 Déploiement de l'IA

L'IA a été l'un des sujets les plus délicats. Exécuter un modèle localement dans Kubernetes peut être coûteux en RAM et CPU. Le modèle Ollama peut être lourd pour l'infrastructure OpenStack.

Plusieurs stratégies ont été envisagées :

1. **Ollama dans Docker Compose local** pour valider la fonctionnalité.
2. **Ollama dans Kubernetes** si les ressources le permettent.
3. **Ollama sur Azure Container Apps** avec min replicas à 0 pour réduire les coûts.
4. **Service IA séparé** pour éviter de surcharger le cluster principal.
5. **Approche hybride** : backend principal dans OpenStack, IA sur Azure.

Cette stratégie hybride est techniquement intéressante car elle montre une capacité à adapter l'architecture aux contraintes réelles.

8. Gestion des images Docker et incidents liés aux pods

8.1 Construction et publication des images

Pour que Kubernetes puisse lancer les services, les images devaient être disponibles dans un registre accessible, principalement Docker Hub.

Le processus général était :

```
docker build -t <image-name>:<tag> .  
docker push <image-name>:<tag>
```

Puis le manifest Kubernetes utilise cette image :

```
image: docker.io/azizbna/pi-clouddoom-user-service:<tag>
```

8.2 Problème ErrImagePull

Un problème rencontré avec `user-service` était :

```
ErrImagePull
```

Cela signifie que Kubernetes n'arrivait pas à télécharger l'image Docker.

Les causes possibles :

- image inexistante sur Docker Hub ;
- tag incorrect ;
- image privée sans imagePullSecret ;
- faute dans le nom de l'image ;
- problème réseau depuis le nœud Kubernetes ;
- image supprimée ou non poussée.

Dans notre cas, le pod `user-service` utilisait une image de type :

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

La stratégie de diagnostic consistait à :

```
kubectl get deploy user-service -n piclouddoom -o wide  
kubectl get deploy user-service -n piclouddoom -o
```

```
jsonpath='{.spec.template.spec.containers[0].image}'{"\n"}'  
kubectl describe pod <pod-name> -n piclouddoom
```

Puis vérifier que l'image existe réellement dans le registre.

8.3 Rollout et rollback

Pendant les corrections, nous avons utilisé les mécanismes Kubernetes de rollout et rollback.

Exemple :

```
kubectl rollout undo deployment/user-service -n piclouddoom  
kubectl rollout status deployment/user-service -n piclouddoom --timeout=180s
```

Le rollback permet de revenir à une version précédente d'un Deployment. C'est une fonctionnalité importante en production, car elle permet de récupérer rapidement après une mauvaise image ou une mauvaise configuration.

Cependant, un rollback ne garantit pas toujours que le service va redevenir disponible. Si l'ancienne configuration dépend toujours d'une ressource indisponible, d'une image manquante ou d'une mauvaise variable d'environnement, le pod peut rester non prêt.

8.4 Analyse des pods non Ready

Quand un pod reste en `0/1`, cela signifie généralement que le conteneur n'est pas prêt ou ne démarre pas correctement.

Les commandes importantes sont :

```
kubectl get pods -n piclouddoom  
kubectl describe pod <pod-name> -n piclouddoom  
kubectl logs <pod-name> -n piclouddoom
```

Ces commandes permettent d'identifier :

- erreurs d'image ;
- crash de l'application ;
- variables manquantes ;
- connexion DB impossible ;
- mauvaise URL Keycloak ;
- port incorrect ;
- probe trop stricte ;
- dépendance non prête.

9. Authentification, OAuth et Keycloak

9.1 Rôle de Keycloak

Keycloak était le serveur central d'authentification. Il permettait de gérer les utilisateurs et de déléguer la connexion à des fournisseurs externes.

Le frontend redirige l'utilisateur vers Keycloak. Keycloak gère ensuite l'authentification, puis renvoie l'utilisateur vers le frontend avec un token ou un code selon le flow OAuth.

Les microservices backend valident ensuite les tokens JWT émis par Keycloak.

9.2 Rôles et autorisation

Les rôles utilisés incluaient :

- `ROLE_USER` ;
- `ROLE_ADMIN` ;
- `ROLE_MANAGER` .

Dans Spring Security, il fallait convertir les rôles présents dans le token JWT en autorités Spring. Cela se fait généralement avec un convertisseur JWT personnalisé.

L'objectif était que les annotations comme `@PreAuthorize` puissent fonctionner correctement.

9.3 Différence entre rôles Keycloak et rôles métier

Un point important du projet est la différence entre :

- les rôles de sécurité Keycloak : `ROLE_USER` , `ROLE_ADMIN` , `ROLE_MANAGER` ;
- les rôles ou statuts métier dans l'application : `FREE` , `PREMIUM` , `ADMIN` .

Ces deux concepts ne doivent pas être confondus.

Les rôles Keycloak contrôlent l'accès technique aux endpoints. Les rôles métier contrôlent la logique fonctionnelle de l'application, par exemple un utilisateur gratuit ou premium.

9.4 Problèmes Google OAuth

La connexion Google a posé des problèmes après le déploiement. Même si LinkedIn avait été corrigé, Google continuait à afficher une erreur.

Les causes probables étaient :

- redirect URI incorrecte ;
- domaine public non autorisé dans Google Cloud Console ;
- mauvaise URL dans Keycloak ;

- erreur entre HTTP et HTTPS ;
- mismatch entre URL frontend et callback Keycloak ;
- problème de configuration Cloudflare.

Pour corriger ce type de problème, il faut vérifier trois endroits :

1. **Google Cloud Console** : URI de redirection autorisée.
2. **Keycloak Identity Provider Google** : callback URI utilisée.
3. **Frontend / domaine public** : URL réellement utilisée par l'utilisateur.

9.5 Problèmes LinkedIn OAuth

LinkedIn a également nécessité une correction. Une fois les bonnes URLs configurées, la connexion LinkedIn a été stabilisée.

Le principe était le même : la redirect URI configurée chez LinkedIn doit correspondre exactement à celle utilisée par Keycloak.

9.6 Problèmes GitHub OAuth

Nous avons ensuite travaillé sur la connexion GitHub. Lors du clic sur "Continue with GitHub", une erreur apparaissait.

L'analyse devait suivre la même logique :

- vérifier l'OAuth App GitHub ;
- vérifier l'Authorization callback URL ;
- vérifier la configuration du provider GitHub dans Keycloak ;
- vérifier le domaine public ;
- vérifier si le callback arrive bien à Keycloak ;
- vérifier les logs Keycloak.

9.7 Importance des URLs publiques

Un point central de cette phase est que les intégrations OAuth ne peuvent pas être traitées comme en local.

En local, on utilise souvent :

```
http://localhost:8080
```

Mais en déploiement public, les URLs doivent correspondre au vrai domaine :

```
https://<domaine-public>
```

Toutes les plateformes OAuth vérifient strictement les redirect URIs. Une petite différence peut provoquer une erreur :

- slash final différent ;
- http au lieu de https ;
- mauvais port ;
- mauvais path ;
- mauvais domaine ;
- callback Keycloak incorrect.

10. Exposition publique et Cloudflare

10.1 Objectif

L'application devait être accessible depuis l'extérieur. Cloudflare a été utilisé pour gérer le domaine et l'exposition publique.

10.2 Rôle de Cloudflare

Cloudflare peut intervenir sur plusieurs aspects :

- DNS ;
- proxy HTTP/HTTPS ;
- certificats TLS ;
- protection basique ;
- règles de redirection ;
- résolution des domaines.

10.3 Erreur Cloudflare 1033

Une erreur Cloudflare 1033 est apparue après un reboot. Ce type d'erreur indique généralement un problème entre Cloudflare et l'origine, par exemple :

- tunnel Cloudflare non actif ;
- service d'origine arrêté ;
- mauvaise configuration DNS ;
- IP d'origine non disponible ;
- cluster ou ingress non démarré ;
- port non exposé ;
- application non joignable.

La démarche correcte est :

1. vérifier que les pods sont en Running ;
2. vérifier que les services Kubernetes sont exposés ;
3. vérifier l'ingress ou le tunnel ;

4. vérifier Cloudflare DNS ;
5. vérifier que l'origine répond localement ;
6. redémarrer le tunnel si nécessaire.

10.4 Impact sur OAuth

Cloudflare et le domaine public ont un impact direct sur OAuth. Si l'URL publique change ou si le service n'est pas accessible via HTTPS, les providers OAuth peuvent refuser la connexion.

Il fallait donc aligner :

- frontend ;
- Keycloak ;
- Cloudflare ;
- Google ;
- LinkedIn ;
- GitHub ;
- backend Spring Boot.

11. Intégration IA et parsing de CV

11.1 Objectif de la fonctionnalité IA

Une fonctionnalité importante du projet était l'analyse automatique du CV. Lorsqu'un utilisateur upload un CV PDF, le système devait extraire des informations utiles et enrichir automatiquement son profil.

Les données visées étaient :

- bio ;
- compétences ;
- expériences ;
- formations ;
- projets ;
- certifications si disponibles.

11.2 Upload du CV

Le CV est envoyé au backend sous forme de fichier PDF. Le backend stocke le fichier dans un dossier dédié, par exemple :

`uploads/cvs`

Le service conserve ensuite une URL relative du CV pour permettre au frontend de l'afficher avec une action "View CV".

11.3 Extraction du texte

Avant d'envoyer le contenu à un modèle IA, le backend doit extraire le texte du PDF. Cette étape est importante car le modèle ne travaille pas directement sur le fichier PDF, mais sur le texte extrait.

L'extraction doit être robuste, car les CV peuvent avoir plusieurs formats, langues et structures.

Le système devait reconnaître des sections comme :

- skills ;
- compétences ;
- education ;
- formation ;
- experience ;
- projets.

11.4 Passage de Gemini à Ollama

Au départ, Gemini avait été envisagé pour l'analyse IA. Cependant, plusieurs difficultés sont apparues :

- problèmes d'URL d'API ;
- problème de clé API ;
- limitations de quota ;
- dépendance à un service externe ;
- difficulté de contrôle du coût.

Une stratégie alternative a donc été adoptée : utiliser **Ollama** avec un modèle local, comme

`llama3.2:3b` .

11.5 Avantage d'Ollama

Ollama permet d'exécuter un modèle localement ou dans un conteneur. Cela offre plusieurs avantages :

- meilleure maîtrise de l'environnement ;
- pas de dépendance directe à une API payante ;
- fonctionnement possible en local ;
- intégration simple via HTTP ;
- possibilité de changer de modèle.

11.6 Limites d'Ollama

Ollama demande des ressources importantes. Même un modèle relativement petit peut consommer beaucoup de RAM et être lent sur CPU.

Dans un cluster OpenStack limité, exécuter Ollama directement dans Kubernetes peut poser problème.

C'est pour cela qu'une stratégie hybride a été étudiée :

- garder les microservices principaux sur OpenStack/Kubernetes ;
- déplacer Ollama vers Azure Container Apps ;
- utiliser `min-replicas 0` et `max-replicas 1` pour limiter les coûts ;
- accepter un cold start plus lent lors de la première requête.

11.7 Mapping des données IA

Le modèle IA devait retourner une structure JSON stricte, par exemple :

```
{
  "bio": "...",
  "skills": ["Java", "Spring Boot", "Docker"],
  "educations": [],
  "experiences": []
}
```

Ensuite, le backend mappe ces informations vers l'entité utilisateur :

- `bio` ;
- `skillsJson` ;
- `educationsJson` ;
- `experiencesJson`.

Cette étape est importante pour transformer une réponse IA non structurée en données exploitables par l'application.

12. Monitoring

12.1 Objectif du monitoring

Le monitoring permet d'observer l'état du cluster et de l'application. Dans un environnement distribué, il est impossible de gérer correctement un système sans visibilité.

Le monitoring permet de répondre à des questions comme :

- quels pods sont en erreur ?
- quel service consomme trop de CPU ou RAM ?
- le cluster est-il stable ?
- les endpoints répondent-ils correctement ?
- les services redémarrent-ils souvent ?
- y a-t-il des pics anormaux ?

12.2 Éléments à surveiller

Dans notre projet, les éléments importants à surveiller sont :

- état des nœuds Kubernetes ;
- état des pods ;
- redémarrages des containers ;
- consommation CPU ;
- consommation mémoire ;
- logs des microservices ;
- disponibilité de PostgreSQL ;
- disponibilité de Keycloak ;
- disponibilité du frontend ;
- erreurs HTTP ;
- erreurs d'authentification ;
- erreurs OAuth ;
- erreurs réseau.

12.3 Commandes de diagnostic Kubernetes

Même avant une stack complète Prometheus/Grafana, plusieurs commandes Kubernetes ont été utilisées pour le monitoring de base :

```
kubectl get nodes
kubectl get pods -n piclouddoom
kubectl get deploy -n piclouddoom
kubectl get svc -n piclouddoom
kubectl describe pod <pod-name> -n piclouddoom
kubectl logs <pod-name> -n piclouddoom
kubectl rollout status deployment/<deployment-name> -n piclouddoom
```

Ces commandes sont essentielles pour diagnostiquer rapidement les incidents.

12.4 Monitoring applicatif

Pour les services Spring Boot, Spring Boot Actuator peut être utilisé afin d'exposer des endpoints comme :

- `/actuator/health` ;
- `/actuator/info` ;
- `/actuator/metrics` .

Ces endpoints peuvent ensuite être utilisés par Kubernetes pour les probes ou par Prometheus pour collecter des métriques.

12.5 Stratégie Prometheus/Grafana

Une stratégie classique de monitoring Kubernetes repose sur :

- **Prometheus** pour collecter les métriques ;
- **Grafana** pour visualiser les dashboards ;
- **Alertmanager** pour envoyer des alertes ;
- **kube-state-metrics** pour surveiller les objets Kubernetes ;
- **node-exporter** pour surveiller les ressources des nœuds.

Dans le cadre du projet, cette stack représente une solution professionnelle pour démontrer une vraie démarche DevOps.

13. Détection d'attaques

13.1 Objectif

En plus du monitoring classique, nous avons prévu une logique de détection d'attaques ou d'activités suspectes.

L'objectif est de surveiller les comportements anormaux dans l'application ou dans l'infrastructure.

13.2 Types d'attaques ou anomalies à détecter

Les anomalies possibles incluent :

- trop grand nombre de requêtes en peu de temps ;
- tentatives répétées de login ;
- erreurs 401 ou 403 répétées ;
- accès à des endpoints sensibles ;
- scans d'URL ;
- pics de trafic anormaux ;
- pods qui redémarrent après surcharge ;
- consommation CPU/RAM inhabituelle ;
- erreurs répétées côté Keycloak.

13.3 Sources de données

Pour détecter ces comportements, on peut utiliser :

- logs des microservices ;
- logs Keycloak ;
- métriques Prometheus ;
- logs Nginx/Ingress ;
- logs Cloudflare ;
- événements Kubernetes ;

- codes HTTP ;
- nombre de requêtes par IP ou par utilisateur.

13.4 Stratégie technique possible

Une stratégie simple mais efficace consiste à :

1. collecter les logs ;
2. extraire les indicateurs importants ;
3. définir des seuils ;
4. déclencher une alerte si un seuil est dépassé ;
5. afficher l'état dans un dashboard ;
6. éventuellement bloquer ou limiter certaines requêtes.

Exemples :

- si une IP génère plus de 100 requêtes en une minute, marquer comme suspect ;
- si un utilisateur provoque plus de 10 erreurs 401 en 5 minutes, déclencher une alerte ;
- si un endpoint admin est appelé par un utilisateur non autorisé, enregistrer un événement critique ;
- si un pod redémarre plusieurs fois, déclencher une alerte d'instabilité.

13.5 Valeur ajoutée pour le projet

La détection d'attaques montre que le projet ne se limite pas au développement fonctionnel. Elle montre une réflexion sur :

- la sécurité ;
- l'exploitation ;
- la surveillance ;
- la résilience ;
- la qualité de production.

C'est un point fort pour une soutenance, car il montre une vision DevOps/SecOps.

14. Stratégie hybride OpenStack + Azure/Vercel

14.1 Problème initial

Le cluster OpenStack avait des ressources limitées. Héberger tous les composants dans le même cluster pouvait provoquer :

- manque de RAM ;
- pods en CrashLoopBackOff ;
- lenteur générale ;
- difficultés avec Ollama ;
- surcharge du control plane ;
- instabilité des services.

14.2 Principe de la stratégie hybride

La stratégie hybride consiste à ne pas tout mettre dans le même environnement.

Exemple de répartition possible :

- OpenStack/Kubernetes : microservices principaux, PostgreSQL, Redis, Keycloak, Kafka ;
- Azure : service IA/Ollama ;
- Vercel ou autre plateforme frontend : frontend Angular ;
- Cloudflare : DNS, HTTPS, proxy et sécurité.

14.3 Avantages

Cette stratégie permet :

- de réduire la charge sur le cluster ;
- d'améliorer la disponibilité ;
- d'utiliser chaque plateforme pour ce qu'elle fait le mieux ;
- de mieux gérer les coûts ;
- de justifier une architecture cloud réaliste ;
- de faciliter la démonstration finale.

14.4 Cas d'Ollama sur Azure Container Apps

Pour Ollama, une solution envisagée était Azure Container Apps avec :

```
min replicas = 0  
max replicas = 1
```

Cela permet de réduire le coût lorsque le service n'est pas utilisé. Le désavantage est le cold start : la première requête peut être lente car le conteneur doit démarrer.

Un autre point important : si le modèle n'est pas stocké dans un volume persistant ou inclus dans l'image, il peut être retéléchargé au redémarrage. Il faut donc prévoir :

- une image custom avec le modèle préchargé ;
- ou un volume persistant ;
- ou un script d'initialisation contrôlé.

15. Problèmes rencontrés et solutions appliquées

15.1 Problèmes d'image Docker

Problème : certains pods ne démarraient pas à cause de `ErrImagePull`.

Cause possible : image inexistante ou mauvais tag.

Solution : vérifier l'image utilisée par le Deployment, vérifier Docker Hub, corriger le tag ou republier l'image.

15.2 Problèmes de pods non prêts

Problème : certains pods restaient en `0/1`.

Causes possibles :

- application qui crash ;
- mauvaise variable d'environnement ;
- mauvaise connexion DB ;
- Keycloak inaccessible ;
- erreur de port ;
- readiness probe trop stricte.

Solution : utiliser `kubectl logs` et `kubectl describe pod` pour identifier la cause exacte.

15.3 Problèmes OAuth

Problème : Google, LinkedIn ou GitHub ne redirigeaient pas correctement.

Cause : mismatch entre redirect URI configurée et URI réellement utilisée.

Solution : aligner les URLs dans Keycloak, le provider OAuth, Cloudflare et le frontend.

15.4 Problèmes Cloudflare

Problème : erreur 1033 après reboot.

Cause possible : origine indisponible, tunnel arrêté, DNS incorrect ou service Kubernetes non exposé.

Solution : vérifier cluster, ingress/service, tunnel et DNS.

15.5 Problèmes de StorageClass

Problème : aucun StorageClass disponible.

Solution : installer local-path-provisioner.

15.6 Problèmes SSH

Problème : host key changed.

Solution : nettoyer `known_hosts` avec `ssh-keygen -R`.

15.7 Problèmes liés à l'IA

Problème : Gemini présentait des problèmes d'API/quota et Ollama demandait beaucoup de ressources.

Solution : migration vers Ollama et réflexion sur un déploiement hybride via Azure.

16. Commandes importantes utilisées

16.1 Kubernetes

```
kubectl get nodes
kubectl get pods -n piclouddoom
kubectl get deploy -n piclouddoom
kubectl get svc -n piclouddoom
kubectl describe pod <pod-name> -n piclouddoom
kubectl logs <pod-name> -n piclouddoom
kubectl rollout status deployment/<deployment-name> -n piclouddoom
kubectl rollout undo deployment/<deployment-name> -n piclouddoom
kubectl get storageclass
```

16.2 Vérification d'image utilisée par un Deployment

```
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}'{"\n"}
```

16.3 OpenStack

```
openstack volume show <volume-id>
openstack volume snapshot list --volume <volume-id>
openstack volume delete <volume-id>
openstack server list
openstack stack list
openstack stack show <stack-name>
```

16.4 SSH

```
ssh-keygen -f ~/.ssh/known_hosts -R <IP>
ssh -i <key.pem> ubuntu@<IP>
```

16.5 Docker

```
docker build -t <image>:<tag> .  
docker push <image>:<tag>  
docker pull <image>:<tag>
```

17. Ce que cette phase nous a appris

Cette phase de déploiement a permis de comprendre que le passage d'une application développée localement vers un environnement cloud est une étape très différente du développement classique.

Nous avons appris que :

- une application qui fonctionne en local ne fonctionne pas automatiquement en Kubernetes ;
- `localhost` n'a pas la même signification dans un conteneur ou un cluster ;
- les variables d'environnement sont critiques ;
- les images Docker doivent être correctement taguées et publiées ;
- les URLs OAuth doivent être exactement correctes ;
- les logs sont indispensables ;
- Kubernetes nécessite une méthode de diagnostic rigoureuse ;
- la persistance doit être prévue dès le départ ;
- les ressources cloud limitées imposent des choix d'architecture ;
- une stratégie hybride peut être plus réaliste qu'un déploiement entièrement centralisé ;
- le monitoring est indispensable pour comprendre l'état réel du système ;
- la sécurité ne se limite pas au login, elle inclut aussi la détection d'anomalies.

18. Bilan global du déploiement

La phase de déploiement du projet Pi-CloudDOOM a été une phase de transformation complète. Nous sommes passés d'une application développée par modules à une application organisée pour fonctionner dans un environnement cloud-native.

Les principales réalisations sont :

- préparation de l'infrastructure OpenStack ;
- création et recréation d'un cluster Kubernetes ;
- mise en place d'un namespace dédié ;
- déploiement des microservices ;
- utilisation d'images Docker publiées ;
- configuration de PostgreSQL, Redis, Kafka et Keycloak ;
- correction de problèmes de pods ;
- gestion des erreurs d'image ;

- utilisation de rollout et rollback ;
- configuration OAuth avec Google, LinkedIn et GitHub ;
- exposition publique via Cloudflare ;
- analyse des erreurs Cloudflare ;
- intégration de l'IA avec Ollama ;
- étude de l'option Azure pour l'IA ;
- mise en place ou préparation du monitoring ;
- conception d'une stratégie de détection d'attaques ;
- documentation des commandes et procédures.

Cette phase a donc couvert plusieurs dimensions du DevOps moderne : infrastructure, conteneurisation, orchestration, sécurité, observabilité, cloud hybride et automatisation.

19. Conclusion

Le déploiement de Pi-CloudDOOM a été une étape longue mais essentielle. Il a permis de rapprocher le projet d'un environnement réel de production. Nous avons utilisé des technologies modernes comme Docker, Kubernetes, OpenStack, Keycloak, PostgreSQL, Redis, Kafka, Cloudflare, Ollama et Azure.

Le travail réalisé montre une vraie démarche cloud-native : les services sont conteneurisés, orchestrés, configurés, surveillés et exposés publiquement. Les problèmes rencontrés, comme les erreurs OAuth, les pods non prêts, les images introuvables, les erreurs Cloudflare ou les limites de ressources, ont été traités comme des situations réelles de production.

La valeur de cette phase ne réside pas seulement dans le fait d'avoir lancé l'application, mais dans la compréhension complète du processus : comment une application microservices passe du développement local à une infrastructure cloud, comment on diagnostique les erreurs, comment on sécurise les accès, comment on surveille le système et comment on adapte l'architecture aux contraintes réelles.

Ce rapport peut servir de base pour la documentation finale du projet, la préparation de la soutenance et la justification des choix techniques devant un jury.

20. Résumé exécutif pour la soutenance

Pendant la phase de déploiement, nous avons transformé notre application Pi-CloudDOOM en une solution cloud-native basée sur des microservices. Nous avons utilisé Docker pour conteneuriser les services, Kubernetes pour les orchestrer, OpenStack comme infrastructure cloud privée, Keycloak pour l'authentification, PostgreSQL/Redis/Kafka pour les composants techniques internes, Cloudflare pour l'exposition publique, et Ollama/Azure pour la partie intelligence artificielle.

Nous avons rencontré plusieurs problèmes réels de production : erreurs d'images Docker, pods non prêts, problèmes de volumes, erreurs SSH, configuration OAuth complexe, erreurs Cloudflare et limites de ressources. Pour chaque problème, nous avons appliqué une démarche de diagnostic basée sur les logs, les commandes Kubernetes, l'analyse des configurations et les corrections progressives.

La stratégie finale n'était pas seulement de tout déployer dans un seul cluster, mais de réfléchir à une architecture hybride capable de répartir les charges. Les services principaux restent sur Kubernetes/OpenStack, tandis que certains composants plus lourds, comme l'IA avec Ollama, peuvent être déplacés vers Azure pour mieux gérer les ressources et les coûts.

Enfin, nous avons intégré une démarche DevOps complète avec monitoring, observation des pods, préparation d'une détection d'attaques et documentation des procédures. Cette phase nous a permis de comprendre les vrais enjeux du déploiement cloud : disponibilité, sécurité, scalabilité, observabilité, résilience et maîtrise de l'infrastructure.